



**Universidad Central de Venezuela.
Facultad de Ciencias.
Escuela de Computación.
Sistemas Operativos (6004).**



Proyecto #1 Diseño e Implementación de una Shell en Linux

Preparador:

Guillermo Hernandez

Estudiantes:

Almeida Valentina.

CI.32008489

Amselmi Alexandra.

CI.32348685

Matheus Corina.

CI.32104618

Sección 2

Caracas, 11/06/2026.

Gestión de trabajos, procesos en segundo plano y señales:

Para este enfoque con respecto a lo solicitado, se buscó modularizar cada una de las tareas pertenecientes a este trabajo. Por ello, se creó la siguiente estructura con el fin de manejar de la forma más limpia posible los trabajos creados por los procesos anteriormente mencionados por un comando:

Para la petición de la tabla de trabajos, se hizo caso a la sugerencia de las listas enlazadas. Sin embargo, para ello el lenguaje C presentaba la limitación de solo manejar struct, por lo que la realización de la misma debía ser a mano. Luego de consultar páginas como https://ftp.gnu.org/old-gnu/Manuals/glibc-2.2.3/html_node/libc_568.html#SEC579 para tener una idea de una mínima estructura para un trabajo, se decidió simplificar la misma. Esta estructura está declarada en job_list.h en la carpeta de Include. Un job, para efectos de este proyecto, está conformado por: el comando relacionado al job (para cuando requiera mostrar sus detalles era fundamental), el pid asignado por el sistema operativo y, para facilidad de manejo por nuestra parte, se decidió utilizar ids internos asignados por nosotros en la propia lista (para mayor comodidad va desde el 1 hasta el número que tenga la lista de trabajos) y un estado (otra variable fundamental ya que es parte de lo que se debe mostrar). Finalmente, para que esta estructura fuera una lista enlazada, debería tener un apuntador a la dirección del siguiente nodo (el siguiente Job).

Por su parte, debíamos manejar operaciones para las demás solicitudes como se requería, por ejemplo para los procesos en segundo plano. Por ello, se aplicaron en el .h una serie de funciones que se desglosaron en la carpeta src, específicamente en job_list.c, donde se tomó inspiración para realizar este tipo de estructuras, además de los conocimientos adquiridos en otras materias (como AYED): <https://www.geeksforgeeks.org/c/c-program-to-implement-singly-linked-list/> Para cada una de ellas se realiza una implementación que se describe a continuación:

Job* Buscar_job_porpid(pid_t id, Job* lista_jobs): Esta función devuelve un trabajo (el que se está buscando). Recibe su pid asignado, lo compara con cada pid de la lista asignada y, si hay una coincidencia, devuelve el job de la lista que arrojó la coincidencia; de otra forma devuelve NULL. Se buscó este enfoque ya que para las señales se requería realizar una búsqueda por pid y por un id interno.

Job* Buscar_job_porid(int id_job, Job* lista_jobs): Realiza lo mismo que hace la anterior función, la diferencia es que compara con el id interno asignado por nosotros.

void Insertar_job(Job cabeza, pid_t id_proceso, const char* comando, int estado): Su enfoque no fue tan complicado ya que este inserta al final de la lista. Para la asignación del id interno, ya que la cabeza inicia en uno, al siguiente simplemente se le asigna el id anterior + 1 con el fin de simplificar las inserciones y las búsquedas. Este implementa un bucle que sigue hasta que el nodo tenga como siguiente un apuntador nulo (llegamos al final); por ello se inserta un nuevo nodo, creando con malloc() la nueva memoria dinámica para el mismo y asignando los

atributos pertinentes, dejando su siguiente con un valor nulo. Para evaluar el comportamiento y manejo de la asignación de memoria se consultó: <https://man7.org/linux/man-pages/man3/malloc.3.html>

`void Eliminar_Job(Job cabeza, int id_job):` Por comodidad se decidió implementar el enfoque de eliminar un job según el id, y así simplemente hacer la búsqueda. Una vez detectado el anterior trabajo al que buscamos, asignamos su siguiente al nodo posterior al objetivo. Luego lo liberamos usando `free()` para que no quede rastro en la RAM, al igual que su comando (ya que este lo almacenamos con `malloc()` lógicamente también debe ser liberado con el fin de prevenir fugas de memoria). Para consultar la liberación segura de bloques de memoria se revisó: <https://man7.org/linux/man-pages/man3/free.3.html>

`void CambiarEstadoJob(Job* cabeza, int id_job, int estado_nuevo):` Se busca el nodo según su id interno y luego se le cambia el estado (los cuales se definieron mediante una leyenda numérica estricta para efectos de este proyecto: Ejecutando = 1, Detenido = 2, Terminado = 3).

`void LimpiarJobs(Job cabeza):` Libera toda la lista de los procesos. Este se implementó para el builtin `exit`; simplemente libera cada puntero de la cola preservando el siguiente en una variable temporal hasta que el siguiente sea nulo.

`void MostrarJobs(Job* cabeza):` Imprime según el tipo de datos cada atributo de cada job en la lista hasta que el siguiente sea nulo. Se implementó ya que se requería para el builtin `jobs` que se solicitaba.

Para efectos de los builtins se usó la mayoría de las funcionalidades de la lista con ciertos cambios, de forma que para el manejo tanto de `fg` como para los procesos en segundo plano se investigó tanto los comandos que se iban a imitar como la estructura base de cómo debería ser, la cual se adaptó a este proyecto en la página: https://sourceware.org/glibc/manual/latest/html_node/Foreground-and-Background.html De forma que se realizaron en `builtins.c` las siguientes funciones (declaradas en `builtins.h` también):

`int builtin_jobs(Job* cabeza):` Primero actualiza el job que va a mostrar actualmente con un bucle que recorre la lista y usa `waitpid()` con el flag `WNOHANG` para determinar el estado en el kernel de forma no bloqueante y capturar a los ya terminados (estado 3) y a los detenidos (estado 2). Para asegurarnos de actualizar la lista antes de mostrarla, se llama a `MostrarJobs()` que dijimos en el anterior módulo y así cumplir con lo solicitado: <https://www.man7.org/linux/man-pages/man2/waitpid.2.html>

`int builtin_fg(Job cabeza, int id_job):` Para ello, según la solicitud que se encuentra en el último párrafo, se busca con la función `Buscar_job_porid()` mencionada anteriormente. Si se encontró, entonces necesitábamos hacer que el hijo continuara, pero aun así poder cambiar su estado y traerlo a la pantalla. Una vez encontrado en la tabla, usamos la función `kill()` más la señal `SIGCONT` para que así continúe y este quede en pausa (no se termina, ya que `kill()` se

puede usar para enviar señales a un cierto pid, en este caso esto le indica que si está detenido puede continuar). Para evaluar la reactivación de procesos suspendidos mediante señales de control se consultó: <https://man7.org/linux/man-pages/man7/signal.7.html> Cabe destacar que para mantener la consistencia de las señales, esta función actualiza la variable global de primer plano con el pid del hijo antes de ceder el control al waitpid(). Para la sintaxis de envío de señales se utilizó: <https://www.man7.org/linux/man-pages/man1/kill.1.html>

`void builtin_exit(Job cabeza):` Una vez que se ejecuta este comando se debe cerrar la shell liberando todas las estructuras usadas. Por ello usamos la señal `kill()` con `SIGKILL` para cada proceso activo con el fin de cerrar los pids de los procesos como tal en el sistema operativo y luego `LimpiarJobs()` para liberar la estructura completa de la lista enlazada evitando dejar procesos huérfanos y zombis en segundo plano. Para consultar su comportamiento se vio: <https://www.man7.org/linux/man-pages/man3/exit.3.html>

`int builtin_bg(Job* lista_jobs, int id_job):` Una vez que se identifica que se requiere reanudar un proceso en el segundo plano, se busca el trabajo detenido dentro de la tabla mediante su id interno. Se le envía la señal `SIGCONT` a su PID utilizando la llamada `kill()` para que despierte y continúe ejecutándose de fondo sin adueñarse de la terminal. Inmediatamente se actualiza su estado numérico interno a 1 (Ejecutando) usando la función `CambiarEstadoJob()`, permitiendo que la shell imprima su progreso y el usuario mantenga el control del prompt de forma síncrona.

`int builtin_cd(Comando* comando_actual):` Se presentaba una situación similar con esta función, la cual representa una característica básica que debe manejar cualquier shell. Aunque no estaba presente de forma explícita en el enunciado del proyecto, en los ejemplos prácticos proporcionados sí se incluía. Por ello, se investigó la lógica más fundamental que posee este comando, la cual consiste en cambiar de carpeta, contemplando opciones esenciales como: `cd` (para regresar al directorio base o HOME del usuario), `cd -` (para regresar al directorio de trabajo anterior) y `cd [más la ruta]` (para navegar a una ubicación específica). Para lograr esto, primero se abordó el problema físico de cómo cambiar de carpeta en el disco. Como se había visto en las clases teóricas, el único capaz de alterar el directorio de trabajo de una terminal es el propio sistema operativo a través del proceso padre. Al buscar la manera de implementar este comportamiento en C, se encontró la llamada al sistema `chdir()`, la cual se utilizó para modificar la posición de la shell de manera directa: <https://www.man7.org/linux/man-pages/man2/chdir.2.html> Ahora bien, para la opción de regresar de forma automática cuando el usuario no pasa argumentos, el sistema simplemente lo redirige a la ruta almacenada en la variable de entorno HOME recuperada con `getenv()`. Mientras tanto, para la funcionalidad de regresar un paso atrás (`cd -`), se tuvo que implementar una lógica distinta: el uso de dos variables de respaldo. Antes de cambiar la dirección actual, el programa almacena la ruta de origen mediante `getcwd()` en una variable para luego asignarla a la variable de entorno `OLDPWD`, mientras que la nueva ruta absoluta se guarda en `PWD` utilizando la llamada `setenv()`. De esta manera, el entorno de la shell se mantiene completamente sincronizado. Para ello se consultó:

<https://www.man7.org/linux/man-pages/man3/getcwd.3.html>

<https://www.man7.org/linux/man-pages/man3/setenv.3.html>

Por otro lado, en el proceso de desarrollo la parte de las tuberías requería que las firmas fueran distintas, por ello se cambió la firma de ciertas funciones con sus respectivos valores de retorno ya que el problema radica en que si se intenta hacer una comparación lógica con estos tipos no podrá compilar, por ello se hicieron las siguientes modificaciones:

1. `int builtin_cd(Comando* comando_actual)`: Se modificó para que retorne 0 si la llamada al sistema `chdir()` tiene éxito al cambiar de carpeta, y 1 en caso de que la ruta no exista o fallen las variables de entorno al leer con `getcwd()`.
2. `int builtin_fg(Job cabeza, int id_job)`: Ahora se encarga de retornar el estado real de salida del proceso que se trajo al frente utilizando la macro `WEXITSTATUS` (si este termina de forma normal), o 0 si el proceso es suspendido nuevamente por una señal capturada por el `waitpid()`.
3. `int builtin_bg(Job* lista_jobs, int id_job)`: Se configuró para que retorne 0 al reanudar y registrar correctamente el trabajo en el segundo plano, y 1 en caso de que el identificador del trabajo proporcionado no exista en la lista enlazada.
4. `int builtin_jobs(Job* cabeza)`: Se le añadió un retorno constante de 0 para mantener la sintaxis homogénea de los comandos evaluados en el despachador.

Para el manejo de señales como `Ctrl+C` y `Ctrl+Z`, se hizo uso de `sigaction()` como habíamos visto en clase. Al aplicar los manejadores se siguió la misma tónica para evitar un cierre abrupto de la shell y, sobre todo, tener el dominio de los procesos que son detenidos en primer plano para que esta acción fuera registrada inmediatamente en nuestra tabla de trabajos. Para ello se implementó la función `Manejadores_senales()` y el manejador llamado `capturar_senal()`, usando la documentación de las páginas:

<https://pubs.opengroup.org/onlinepubs/9699919799/functions/sigaction.html>

<https://man7.org/linux/man-pages/man2/sigaction.2.html>

Durante el desarrollo de este último módulo de señales se nos presentó un problema: el manejador `capturar_senal` no guarda contexto, ni del comando que escribió el usuario en la consola, ni qué PID exacto se está ejecutando en primer plano en ese instante, dejando la función sin datos para actualizar la lista. Para solucionar esto, se nos ocurrió diseñar un mecanismo de comunicación interna basado en variables globales calificadas con el modificador `volatile` (`pid_primer_plano` y `comando_primer_plano`), asegurando que el compilador no optimice el acceso a la memoria RAM durante interrupciones asíncronas conforme lo que dice: <https://en.cppreference.com/w/c/language/volatile> De esta manera, justo antes de que el `main.c` o el builtin `fg` ejecuten un comando en primer plano, respaldan allí de forma síncrona el PID del hijo y su nombre real. Al presionarse `Ctrl+Z`, el manejador ahora sí cuenta con la información necesaria en memoria estática para interceptar al hijo, pausarlo limpiamente con `kill(pid, SIGSTOP)` mediante el envío de la señal de detención forzada `SIGSTOP` e insertarlo o actualizarlo en la lista de trabajos usando el estado 2.

otras paginas consultadas fueron:

<https://man7.org/linux/man-pages/man2/wait.2.html>

<https://man7.org/linux/man-pages/man3/exec.3.html>

<https://man7.org/linux/man-pages/man3/sigsetops.3.html>

<https://man7.org/linux/man-pages/man2/sigaction.2.html>

Historial de comandos previos:

Esta es la primera sección que se ejecuta dentro del bucle while del inicio, inicialmente depende de dos valores numéricos, para evitar ciertas complicaciones se decidió trabajar con un número de 128 comandos máximos que puede cargar el historial en el arreglo local y 1024 caracteres por tamaño de línea como máximo, este último haciendo alusión a que las shell reales poseen un cierto rango máximo de escritura del comando, y en este caso se decidió usar una potencia de dos para rápido procesamiento. Cabe acotar que es un archivo oculto por lo que su nombre es “.historial_ucevshell”, que se asemeja mucho al archivo real que se puede observar usando “ls -la” de la bash de Linux y otros archivos del mismo tono.

Una vez definido mi arreglo de cadenas de caracteres que va a fungir como mi historial, llamado en este caso “arreglo_memoria” aquí se recarga esa porción del historial de los últimos 128 comandos utilizados el cual se va a ir llenando y una vez llegue al tope, se va a rodar todo un espacio a la izquierda en el arreglo para abrir espacio y siempre tener únicamente los más actuales, decisiones tomadas para agilizar el llenado; además se tiene una variable entera de contador para saber cuántos están en el historial actualmente.

La función principal para realizar estas funciones descritas es la de “carga_inicial”, ella, dada la ruta raíz de la máquina que se esté utilizando en ese usuario, va a abrir un archivo en modo de solo lectura, verifica si este no es nulo, es decir, si no abrió correctamente, y posterior a ello comienza a extraer las líneas de texto del archivo historial actual usando fgets(), mientras la cantidad que se ha guardado no supere la máxima definida (ya que debía reservar la memoria tomando en cuenta una cantidad definida) entonces se va a ingresar la cadena en la última posición con respecto a donde está ubicada actualmente el llenado y lo va a colocar, posteriormente cierra la cadena con el caracter NULL para finalizar la construcción de la cadena y aumenta la cantidad del total registrado.

La segunda función que es más un auxiliar, es la de “reiniciar_arreglo_historial” donde sí se llegó al tope de mi arreglo local que contiene el historial, simplemente vuelvo a repetir la estrategia de mover todo un espacio a la izquierda e incluir al final el nuevo comando.

Finalmente por esta sección se tiene “editar_historial”, esta se encarga de todo lo relacionado con la parte de incluir un nuevo comando en el historial de archivo general y en la versión local que se está manejando, esta función recibe de parte del main tanto la ruta raíz como una cadena de caracteres a la cual ya se ha dado enter y posteriormente será procesada por la sección de código de parseo, arreglos e inicialización de banderas. Esta inicia abriendo el archivo en modo append, que como vimos en clase y los laboratorios, es muy útil para indicar que el proceso de escritura del archivo será a partir del final del archivo actual, de esa manera evitando que el historial se sobrescriba siempre desde el inicio o desde la posición donde se deje el puntero de escritura. Posteriormente, escribe en el archivo la línea a la cual

se le ha aplicado el enter y verifica si puede guardarlo de una en el arreglo y si no, vuelve a aplicar la estrategia de mover todo un espacio a la izquierda.

Cabe acotar que posee un archivo .h el cual permite que se conecte con las función main y el definir las variables globales necesarias, al igual que la firma de las funciones.

Modo No-Canónico y movimiento con flechas:

Todo este módulo se encuentra comprimido en un archivo flechas.c que engloba todas estas funcionalidades, además de un archivo de cabecera que posee las firmas de las funciones para que puedan ser llamadas y usadas desde el main y en las signals.

La primera función que se posee es “habilitar_no_canonico”, que como su nombre da a entender, es la función encargada de habilitar el modo no canónico, en este punto es relevante aclarar que es el modo no canónico, este es también llamado “raw” como se describe en el pdf de instrucciones y en la bibliografía consultada para esta parte, se refiere al modo de operación de una shell en el cual cada dato ingresado se va a ir enviando al programa de forma casi inmediata para un preprocesamiento, sin esperar a que el usuario realice click en Enter, al activar este modo se busca principalmente desactivar todas las banderas relacionadas con este proceso, para permitir la edición de la línea de comandos en tiempo real, como operaciones de borrado, inserción al final de la línea, inserción en algún lugar intermedio de la línea o realizar Enter en cualquier posición de la línea y agarre todo el comando como válido.

Tomando las recomendaciones del instructivo, lo visto en clase y lo investigado en los siguientes enlaces de forma principal:

<https://www.ibm.com/docs/es/aix/7.3.0?topic=files-termiosh-file> ,

<https://www.ibm.com/docs/es/aix/7.3.0?topic=t-tcgetattr-subroutine> ,

<https://www.ibm.com/docs/es/aix/7.3.0?topic=t-tcsetattr-subroutine>.

Se inicia definiendo la estructura de términos que se va a utilizar y posteriormente utilizo tcgetattr(), la cual va a buscar todos los atributos actuales del modo general en que se encuentra la terminal en el descriptor 0 o por defecto determinado por STDIN_FILENO, sin embargo lo más importante es lo siguiente, utilizando el modo actual se genera una copia a la cual se va a acceder a su atributo correspondiente a las banderas y se van a modificar o en todo caso, apagar 2 en específico, la primera es ICANON y la segunda ECHO, la primera consiste en indicar si la terminal se encuentra en modo canónico o no canónico, al desactivarse pasa a esta en estado no canónico, quitando la necesidad del enter para ir enviando los caracteres de entrada; la segunda se usa para desactivar lo que la bibliografías llaman eco de caracteres, que básicamente es el hecho de mostrar en tiempo real los caracteres que escribe el usuario en la ventana de la terminal, al estar desactivada, no va a mostrar visualmente lo que se está escribiendo, sino que se va a ir almacenando en un contenedor, un buffer, que se explicará en la función correspondiente, de esa manera, se puede controlar que se imprime, cuando y con que modificaciones se muestra. Ambas en conjunto resultan muy útiles al momento de querer programar posteriormente el procesamiento de las 4 flechas de dirección, la edición y borrado, ya que ciertos caracteres debería interceptarlos y procesarlos en vez de imprimirlos. Finalmente se colocan los parámetro nuevamente en el struct con tcsetattr().

La función de “habilitar_canonico”, sólo se encarga de resetear el modo que fue previamente alterado por la función descrita anteriormente, es decir se vuelve a usar modo en vez de modo2, para ello utiliza STDIN_FILENO para indicar el descriptor por defecto volviendo a modo a su estado original y además se tiene TCSAFLUSH que se encarga de esperar a que el programa culmine cualquier operación de escritura que estuviera haciendo y lo envía a la pantalla, además de vaciar el buffer utilizado cuyos datos no hayan sido leídos o impresos.

La función de borrar es una auxiliar para desplazar el cursor un espacio a la izquierda, borrar lo que se requiere sustituyéndolo por un espacio y volver a mover visualmente el cursor de impresión sobre ese espacio vacío, esto recibe la posición actual del puntero de posición, el cual debe decrementar en uno al dirigirse hacia la izquierda con \b.

La función principal de esta sección es la de leer_actual(), su función es atrapar los caracteres leídos antes de que se presione el enter e ir interceptando todas las operaciones del modo no canónico necesarias, fueron modeladas las cuatro flechas de direcciones, el borrado, la inserción y modificación, inició habilitando el modo no canónico ya que esta función es llamada en main, posteriormente utilizo memset para inicializar todo el espacio de cierto tamaño con 0 (bibliografía consultada: <https://www.ibm.com/docs/es/i/7.5.0?topic=functions-memset-set-bytes-value>) e iniciamos la cadena actual como vacía por defecto, luego activamos un ciclo infinito el cual solo será roto por algún error o porque se presione enter, este va a ir usando el getchar() para obtener caracter por caracter y compararlo con las siguientes opciones:

- 1- Si es el enter, entonces debe terminar de cerrar la cadena con el carácter nulo, hacer un salto de línea y devolver el modo a canónico.
- 2- Si se da click al botón de borrado y se recibe el caracter \b, traducido como 127, es porque se quiere borrar un carácter, este tiene dos vertientes, la primera si estamos en la posición final de la línea simplemente llama a la función borrar(), y se decrementa la posición, en este caso no es necesario redibujar visualmente el cursor de escritura, pero si se necesita imprimir la línea ya que sin el ECHO activado no se muestran. La segunda es que intente borrar en algún lugar intermedio, en todo caso, simplemente se usará la estrategia de tener un bucle que incrementa la posición y mueva uno a la izquierda todos los caracteres desde la posición hasta el final eliminando el carácter que se deseaba borrar, como esto genera que el cursor se mueva, después de imprimir la nueva cadena y almacenarla con el cambio, se devuelve el cursor de escritura a la posición en que borró y continua.
- 3- Las flechas, estas se identifican primero por captar el carácter de escape, lo cual significa que si lo lee es muy probable que lo siguiente sea un corchete y una letra (A,B,C,D) referente a la flecha que van a representar, una vez se comprueban cuidadosamente, se entra a un switch de casos en base a las letras que es el diferenciador, en el caso de la A es la flecha hacia arriba, esta va a buscar la variable que se declaró en el punto de historial para guardar los últimos comandos y los copia como la línea actual, pero para que no se pierda el texto si se ha escrito, se mantiene una variable global para mantener este dato que aún no está en el historial, modificó la cantidad actual de caracteres y se imprime lo recuperado por el historial, luego se hace un continue para que no continue y entre en otra iteración. Ahora la B, esta es

la flecha hacia abajo, esta posee la restricción de que si estoy en la dimensión actual, es decir la reciente impresión, por ende, verifica primero la posición en el arreglo del historial y luego baja una posición si la condición es productiva e imprime lo que se encuentre en el historial. Con la C se refiere a moverse a la derecha pero sin reescribir por lo cual únicamente se imprime el carácter de movimiento, misma situación se aplica con la flecha D que es la izquierda.

4- Si se quiere insertar, hay dos casos, el primero en el cual estamos en el final de la línea y simplemente se puede colocar el carácter en la posición actual de la cadena, y aumentar las dimensiones y todas las variables auxiliares e imprimir en la terminal el cambio para que lo vea el usuario de manera dinámica; en la segunda, queremos insertar en otra posición que no es el final, en este caso, se optó por la estrategia de mover una posición a la derecha a todos los caracteres que se encuentren después de la posición actual, para abrir un espacio donde insertar este nuevo carácter y posteriormente lo imprime.

Finalmente se reactiva el modo canónico y se culmina la iteración volviendo a empezar el ciclo hasta que ocurra el enter o un fallo.

Otras referencias consultadas y a las que se hace alusión en el código:

<https://share.google/uGluVDYEH5iymHrFZ>

<https://share.google/tUm0FNLYbxOD3QF16>

https://en.wikipedia.org/wiki/ANSI_escape_code?hl=es-VE

Parseo y banderas:

Esto corresponde al documento procesar_comando.c y estructura_comando.h, este archivo corresponde al proceso de parseo de los datos, es decir, como se van a picar, guardar y almacenar los datos, sumado a una serie de banderas y consideraciones. En cuanto a la estructura de almacenamiento se utilizó un arreglo de structs de tipo “Comando”, este fué definido con los siguientes atributos:

```
int hay_tuberia; //después de ese comando hay tubería o no, para control
```

```
int modo; //valores 1,2,3 1 si hay un or , 2 si hay && , 3 si hay ;
```

```
char* r_entrada; //guardar argumento de redirección de entrada
```

```
char* r_salida; //guardar argumento de redirección de salida
```

```
int orden; //1 si entrada, 2 si salida
```

```
char* instruccion; //arreglo con la instrucción base
```

```
char** argumentos; //arreglo de cadenas de caracteres para cada argumento que tenga ese comando
```

```
int cant_argumentos; //solamente por facilidad de guardar el número
```

```
int bandera_2plano;
```

A forma de acotación, se consideró que aunque la instrucción principal como puede ser un cd, también es un argumento, pero para facilidades del módulo de ejecución se decidió separar en su propio campo de cadena de instrucción y todo lo demás como argumentos en un arreglo, en cuanto a las banderas, como se cuenta con varios operadores, cada bandera corresponde a la presencia de un operador distinto, a manera de que el ejecutor tenga un fácil acceso a ello y no deba hacer verificaciones directamente del operador sino sólo de variables

numéricas más rápidas de comparar, estas se le atribuyen al comando a la izquierda del operador para identificarlo, como son operaciones binarias, y la mayoría tienen condiciones donde la ejecución del comando izquierdo condiciona si el derecho se ejecuta o no, así que cada bandera fue considerada para hacer comparaciones tempranas para que el ejecutor pueda operar más rápidamente.

Primero tenemos la función auxiliar de limpiar(), la cual se va a encargar de la limpieza de un caracter no imprimible en una cadena de texto para sustituirla por un espacio en blanco, esto a causa del uso de strtok(), como se definieron los delimitadores usando espacios en blanco, en caso de haber unas comillas simples o compuestas, el comando fallaba y omitía el espacio en blanco y eso generaba errores en el ejecutor, por ello, investigando, la estrategia que se podía utilizar era sustituir parcialmente el espacio por un caracter no imprimible que es un caracter que no se permite usar en nombres de carpetas y archivos en la mayoría de casos, esta parte de cambiar por el carácter no imprimible se hace en la función de procesar_c, esto ayuda a solventar el problema y ya luego se deshace el cambio para evitar fallos.

La segunda función auxiliar, se usa error_o_liberar() , se usa para liberar todas las variables que reservaron memoria dinámica durante la ejecución del parseo, para que de esa forma no se mantenga en memoria ocupando espacio y por buenas prácticas, siempre que se reserva memoria dinámica al final debe ser liberada. Así que esta función se llama cuando ocurre un error para liberar toda la memoria o simplemente al terminar con éxito.

Llegando a la función principal procesar_c(), se hace una copia de la línea pasada por el main, para hacer primero un preprocesamiento en el cual, primero voy buscando caracter por caracter hasta encontrar si hay o no comillas, si las encuentra, verifica si hay alguna comilla de salida, entonces en base a lo que se encuentre entre las comillas, va a verificar si hay algún espacio y lo sustituye por el carácter no imprimible, esto aplica para comillas simples y dobles. Sino produce un error que fue realizado con un color distintivo rojo para llamar la atención.

Posterior a este preprocesamiento se usa la variable de la línea original para un ciclo donde se va a identificar la cantidad total de operadores que hay, al ser binarios, de ese cálculo se puede concluir que la cantidad que presente incrementada en uno, es la cantidad de sub comandos dentro del comando general que voy a trabajar, en este caso, ese número se usa para armar el arreglo de tipo Comando y reservar la memoria dinámica que voy a utilizar. Ahora se ingresa a un bucle que solo será detenido al llegar al final de la cadena, es decir, el nulo al final, por ello es que en etapas anteriores era tan importante encargarse de cerrarla. Luego de hacer las verificaciones de obtener una palabra nula y de que no se empiece con un carácter de operador, finalmente se usa la función strtok() la cual va a picar la cadena de caracteres hasta cierto delimitador, que en este caso es el espacio o tabulador, se limpia la palabra y al ser la primera es una instrucción, se inicializan los campos del struct por defecto e inician las comprobaciones hasta que se acabe la palabra picada por strtok y este devuelve NULL, dentro del bucle de argumentos, se busca primero verificar si la bandera “cambio” está encendida, es decir, anteriormente leyó un operador, porque debe descartar un error de tener más de un operador no consecutivo como por ejemplo usar “&& &” o cosas de ese estilo, si no tiene nada de eso, se procede a avanzar en el arreglo de struct y almacenar la

instrucción para posteriormente inicializar los campos del struct, devuelvo la bandera a su estado original y continuo a la siguiente iteración para iniciar con sus argumentos.

Si no entra en este if, se procede a hacer una comprobación individual para saber si el carácter leído es un and, or, pipe, redirección de entrada o de salida, y en base a ello se encienden las banderas y se rellenan los parámetros adecuados dependiendo del caso, y además cambia la bandera de “cambio” a 1, y limpia la palabra en todo caso de que se vaya a guardar para que no imprima el carácter no imprimible. En el caso de las redirecciones prenden la bandera dependiendo de cual se encendió o cual se guardó.

Todo lo demás que no entre en estos campos, significa que es o un argumento normal, entre comillas o que posee el operador de background &; primero se obtienen las longitudes y valores necesarios para discriminar, luego inspecciona si el último carácter antes del nulo es el operador de background '&', en ese caso tengo que verificar si el comando actual es el último de la cadena o no, porque solo puedo mandar al background si el operador está hasta el final, luego de confirmar ello, tengo dos vertientes, la primera es que esté completamente solo al final y simplemente enciendo las banderas pertinentes o el segundo donde esté conectado al argumento que iba antes de él, en este caso se debe cortar el operador para que el argumento quede como debe estar, limpiar la cadena del carácter no imprimible y guardar el argumento. Si todo eso falla, significa que tengo combinaciones de operadores en algún lugar que son completamente inválidas como “&&&&&&”, y entonces los intercepta y da error.

Luego verifica si hay comillas simples o dobles y si estas están debidamente cerradas, si lo están, rueda todo un espacio a la derecha y sustituye la última comilla por el carácter nulo para cerrar la cadena, decrementando el tamaño en 2, y asegurándose de que las comillas no estorben al ejecutor, a su vez, luego hace una limpieza del carácter no imprimible y se almacena como argumento, esto pasa tanto para simples como para dobles comillas. Si todos los anteriores fallan, entonces es un argumento normal y sencillo, entonces simplemente lo limpia y luego lo guarda donde corresponde y aumenta la cantidad de argumentos.

Para culminar, si la bandera de cambio sale del while encendida eso genera un error ya que no se puede dejar al final un operador que no sea background de último, así que lo marca como error, si no entra a ese if, simplemente cierra con NULL el final de los argumentos del comando y retorna la estructura comandos completa que se utiliza en el main.

Ejecución de comandos:

Este módulo se encuentra desarrollado en ejecutar.c, su objetivo es encapsular toda la lógica necesaria para ejecutar comandos, identificando los diferentes tipos de instrucciones que pueden ser ingresadas por un usuario llamando así a las funciones pertenecientes a los módulos dedicados a manejar cada caso en específico (pipes.c, path.c y redireccion.c, explicados más adelante, además de builtins.c y job_list.c que fueron explicados anteriormente).

El módulo cuenta con cuatro funciones: `int es_builtin(char *instruccion)`, `ejecutar_en_hijo(Comando *cmd)`, `ejecutar_uno(Comando *cmd)` y `void ejecutar_comando(Comando *comandos, int numero)`

El flujo del programa comienza cuando `main.c` llama a `ejecutar_comando()`, el cual recibe un arreglo de tipo `Comando` relleno anteriormente en el parser, y que incluye la información necesaria para identificar cada tipo de instrucción. Esta función recorre el arreglo de comandos con el número de comandos ingresados. Primero llama a la función `es_builtin()` la cual comprueba con `strcmp` si el nombre de la instrucción coincide con un built-in implementado, si ese es el caso retorna 1 y se pasa a identificar el built-in exacto en `ejecutar_comando()` para llamar a la función correcta en `builtins.c`, y avanzar al siguiente comando.

Ahora, si la instrucción no se trata de un built-in, se comprueba si se trata de una cadena de pipes, si ese es el caso avanza en el struct con un iterador copia llamado `fin` en busca del último comando de la cadena de pipes, una vez lo encuentra realiza `int codigo_salida=ejecutar_cadena_pipes(comandos, i, fin)`; esta función pertenece a `pipes.c` y devuelve el código de salida para manejar pipes correctamente con operadores.

Si la instrucción no contiene tuberías se realiza `int codigo_salida=ejecutar_uno(&comandos[i])`; que también devuelve el código de salida para manejar correctamente operadores. En esta función primero llama a `buscar_en_path(cmd->instruccion)` implementada en `path.c` y guarda la dirección que retorna esta función, si la función devuelve `NULL` se trata de un comando que no tiene un binario en `PATH` para ejecutar por lo que se realiza un `exit(1)`. Si existe el binario se reserva suficiente memoria con `malloc()` para los argumentos del comando, el nombre del comando y un `NULL`. Se crea un proceso hijo para ejecutar el comando, si `fork()` falla se imprime un mensaje de error y retorna -1. Si `fork()` no falla en el proceso hijo se pregunta si el comando contiene redirecciones, si hay pueden ocurrir únicamente 3 casos hay dos redirecciones de tipo distinto ya sea primero de entrada y luego de salida o viceversa, hay una de entrada o hay una de salida, con una cadena de `if` se identifica cada caso, y se llama a la función correspondiente implementada en `redireccion.c` en el orden correcto de haber las dos. Las redirecciones también pueden fallar así que estas funciones también permiten retornar un código de error e imprimir un mensaje de error. Finalmente se ejecuta el binario con la ruta encontrada y los argumentos guardados, es decir se decidió utilizar `execvp()` y pasarle el path encontrado, aunque sabemos que existe la función `execvp()` que busca automáticamente el binario en el `PATH` y lo maneja, al ya haber implementado e utilizado la búsqueda en `PATH` se consideró que buscarlo nuevamente con `execvp()` no era necesario. Si `execv` falla se imprime un mensaje de error. Una vez de regreso al proceso padre se libera la memoria reservada para la ruta del comando y para los argumentos del comando, además se comprueba si el comando se debe ejecutar en segundo plano, de ser así se agrega a la lista de jobs. Si el comando se ejecuta en primer plano se actualiza el PID a PID del proceso en primer plano para poder manejar señales, se espera a que el proceso hijo termine o se detenga (en caso de `Ctrl+Z`), se reinicia el PID del proceso en primer plano y se retorna el código de salida del proceso hijo.

El manejo de operadores consiste en dos `if` uno para cada operador (`&&` o `||`) si se cumple la condición para que no se ejecute el segundo comando se realiza un `break` al ciclo. Estos `if` se realizan luego de llamar a `ejecutar_cadena_pipes()` o `ejecutar_uno()`, en la función `ejecutar_comando()`.

Nótese que en la función `ejecutar_comando()` nunca se comprueba explícitamente si fue ingresado un `;`; esto se debe a que si fueron ingresados varios comandos se encuentran ya separados en el arreglo tipo `Comando` así que basta con avanzar en el ciclo principal de la función que recorre dicho arreglo.

La última función en este módulo es `ejecutar_en_hijo()` su existencia se debe a cómo se manejan los pipes por lo que es llamada únicamente en `pipes.c`, los pipes crean de por sí un `fork()` mucho antes de ejecutar el comando y `ejecutar_uno()` también realiza un `fork()`, si se usara `ejecutar_uno()` el comando se ejecutaría en un proceso nieto, realizaría un `dup2()` y sobrescribiría el `dup2()` ya realizado en pipes. También se decidió manejar las redirecciones en `pipes.c` por lo que esta función no las contiene. Estas son las únicas diferencias entre las dos funciones, de resto al igual que en `ejecutar_uno()`, `ejecutar_en_hijo()` busca en `PATH`, reserva memoria para `args` y llama a `execv()`.

Comportamiento en operadores y background:

Cuando se usa un `&` al final de una instrucción que contiene `&&` o `||`, se puede observar un comportamiento indefinido, esto se debe a que el proceso va a background inmediatamente y no se espera su código de salida por lo tanto el código de salida se asume como 0 para todos los comandos pertenecientes a la instrucción, al asumirse que el comando nunca falla `&&` siempre ejecuta el siguiente comando y `||` nunca lo ejecuta. Este comportamiento de asumir éxito también se puede observar en algunas shell conocidas.

Pipes:

Este módulo se encuentra implementado en `pipes.c` y se encarga de: Crear y gestionar tuberías entre múltiples procesos, conectar la salida de un proceso con la entrada del siguiente, manejar redirecciones en cadenas de pipes y gestionar procesos en background dentro de pipes.

Para lograrlo, primero calcula la cantidad de comandos con una resta simple ya que recibe la posición de el primer y último comando encadenado con pipes en la instrucción, y crea suficientes descriptores de archivo basado en este cálculo con un arreglo, de igual manera crea PIDs y crea una tubería por cada comando. Si falla la creación de alguna de las tuberías imprime un error y retorna -1 para usar pipes con operadores. Una vez hecho esto crea un hijo por cada comando, si la creación del proceso hijo falla imprimimos un error y también retornamos -1. Cada vez que estemos en un proceso hijo, se comprueba si no es el primer comando para conectar `stdin` al pipe anterior, se comprueba si no es el último comando para conectar `stdout` al pipe siguiente, y luego se cierran todos los extremos del pipe para evitar bloqueos. Una vez hecho esto se manejan las redirecciones igual que `ejecutar.c` y se llama a `ejecutar_en_hijo()`, y una vez ejecutado se cierra el pipe anterior para evitar bloqueos.

Una vez de regreso al padre se manejan los procesos en segundo plano y se captura el status al igual que en `ejecutar.c`, con la diferencia que capturamos y retornamos únicamente el código de salida del último comando del pipe.

Búsqueda de binarios en el PATH del sistema:

Este módulo se encuentra desarrollado en path.c y se encarga de: Buscar el binario de un comando en las rutas del PATH, verificar permisos de ejecución con access(), retornar la ruta completa del binario o NULL si no existe.

Estas tareas son realizadas en una función: char *buscar_en_path(char *instruccion)

Primero obtenemos el PATH del sistema con getenv("PATH") y la guardamos, si el PATH es NULL imprime error y retorna NULL, se realiza una copia de PATH, debido a que getenv() retorna un puntero directo a la memoria del entorno del proceso, y si se modifica, seria un error fatal, el shell del sistema ya no puede encontrar ningún binario y funciones del sistema operativo que dependan del PATH también dejarían de funcionar, si la copia es NULL imprime error y retorna NULL, tokenizamos con separador : que es el que separa las diferentes direcciones que se encuentran en el PATH, y con un for se prueba construyendo la ruta completa directorio/comando con cada directorio posible en cada iteración se verifica si el archivo existe y es ejecutable, retornando así una copia de la ruta encontrada o avanzando al siguiente token. Además se libera la copia del PATH.

Redirecciones:

Este módulo se encuentra desarrollado en redireccion.c y se encarga de: redirigir la salida estándar hacia un archivo (>) o redirigir la entrada estándar desde un archivo (<) gracias a dos funciones una para cada tipo de redirección.

La primera abre el descriptor de archivo en sólo escritura, si no existe el archivo lo crea y si existe lo vacía, si abrir el archivo falla imprimimos un error y retornamos el código de error, redirigimos stdout al archivo abierto y si la redirección falla imprimimos un error. Cerramos el descriptor de archivo y retornamos 0.

La segunda es muy similar, pero abre el descriptor de archivos en solo lectura, si abrir el archivo falla imprimimos un error y lo retornamos, redirigimos stdin al archivo abierto y si la redirección falla imprimimos un error y lo retornamos, cerramos el descriptor de archivo y retornamos 0.

MAIN:

En la función principal main(), primero se incluyen todos los archivos de cabecera para que sirvan de enlace con las llamadas a los otros .c de los módulos de funcionamiento, este mismo toma como variables externas todas las firmas globales de funciones o variables de los .h que realmente requiera llamar. Primero inicializa su lista de jobs y llama a la función getenv(HOME) para obtener la ruta del directorio raíz y concatenar el nombre del archivo, para que si no existe lo crea y si existe no lo hace; esta ruta se usará para buscar el historial y cargarlo en la memoria local del programa, usando las funciones ya descritas en páginas anteriores del informe, una vez hecha la carga, se llama al "manejador_de_señales()" e inicio el bucle infinito del ciclo de vida de la shell, al iniciar imprime el formato de la shell y activa el modo no canónico por medio de leer actual, una vez se haya retornado el valor del enter, entonces verifica si la línea devuelta tiene al menos un carácter y envía todo al historial, no sin antes colocarle el salto de línea, luego define el struct de tipo comando y para rellenarla se envían los parámetros a la función del parser, por lo que al terminar y devolver el struct que

inicializa el del main, se pasa a verificar lo referente a procesos de primer y segundo plano, luego llama al ejecutor del comando y libera la memoria dinámica pertinente; esto se repite en cada uno de los bucles y al finalizar, libera todos los elementos de memoria dinámica restante y retorna cero, terminando la ejecución de la shell.